signus 변경분

바뀐 파일은 통째로 실었습니다 — 그 파일 쪽만 새로 쓰면 됩니다

기준점 2026-08-26 (42d9dd9+수정중) → 2026-08-28 · 44ee8e0+수정중 | 4개 파일 · **+56** / −61 줄 · 코드 24쪽 | 새 코드북 69쪽

바 뀐 내 용

· 웹 버스트 지도가 뭉개져 보이던 원인: 전체 녹음을 72×144 조각으로만 보내 화면에서 크게 늘였기 때문. 이제 서버가 sa 프로브의 PNG 와 똑같은 규칙(hamming 256/128, 바닥 p25, 대비 10~35dB, 열 최대 풀링, 흑백 235)으로 띠를 만들어 보내고, 화면은 그 픽셀을 보간 없이 그대로 확대한다 — sa PNG 를 확대해 보는 것과 같은 그림이 된다.

□	signus/spectrum.py	+18	−17	2쪽	코드북 9쪽
□	signus/pipeline.py	+6	−9	2쪽	코드북 31쪽
□	signus/web/style.css	+12	−3	10쪽	코드북 46쪽
□	signus/web/app.js	+20	−32	14쪽	코드북 51쪽

초록 = 새로 쓸 줄 (오른쪽 숫자가 새 줄번호) · 색 없는 줄 = 그대로 · 붉은 점선 = 그 자리에서 몇 줄 사라짐 · ↳ = 윗줄에 붙는 이어짐 (원래 한 줄이니 붙여서 입력)

signus/spectrum.py +18 −17 · **코드북 9쪽** · 새로 쓸 줄 1, 4, 22–37

```
    1 + """Spectrum + spectrogram-strip views (display only, never used for detection)."""
  2   2
  3   3 import numpy as np
    4 + from scipy.signal import stft, welch
  5   5
  6   6
  7   7 def _db(p: np.ndarray) -> np.ndarray:
  8   8     return 10 * np.log10(p + 1e-20)
  9   9
 10  10
 11  11 def spectrum(x: np.ndarray, fs: float, bins: int = 256) -> dict:
 12  12     """Averaged PSD of the burst, decimated to `bins` points. Frequencies in kHz."""
 13  13     nper = int(min(4096, max(64, x.size // 8)))
 14  14     f, p = welch(x, fs=fs, nperseg=nper, return_onesided=False, detrend=False)
 15  15     f, p = np.fft.fftshift(f), _db(np.fft.fftshift(p))
 16  16     if f.size > bins:  # max-pool so narrow tones survive decimation
 17  17         k = f.size // bins
 18  18         f, p = f[: k * bins].reshape(bins, k).mean(1), p[: k * bins].reshape(bins, k).max(1)
 19  19     return {"f": np.round(f / 1e3, 3).tolist(), "db": np.round(p, 2).tolist()}
 20  20
 21  21
    22 + def strip(x: np.ndarray, fs: float, max_cols: int = 1600) -> dict:
    23 +     """sa 프로브의 PNG 스펙트로그램 띠와 같은 조판의 회색조 이미지 (0..235, row-major,
    24 +     위 행 = +fs/2 쪽). hamming 256/128 STFT, 바닥 p25 + 대비폭 10..35 dB 클램프, 열은
    25 +     최대 풀링으로만 줄인다 -- 보간이 짧은 버스트를 뭉개지 않게."""
    26 +     xd = x.real if np.iscomplexobj(x) else x
    27 +     nper = int(min(256, max(64, 1 << int(np.log2(max(xd.size // 2, 64))))))
    28 +     _, _, z = stft(xd, fs=fs, window="hamming", nperseg=nper, noverlap=nper // 2,
    29 +                    return_onesided=True, boundary=None, padded=False)
    30 +     db = 10 * np.log10(np.abs(z) ** 2 + 1e-12)
    31 +     lo = float(np.percentile(db, 25))
    32 +     rg = float(max(10.0, min(35.0, np.percentile(db, 99.5) - lo)))
    33 +     g = np.clip((db - lo) / rg, 0, 1)[::-1]
    34 +     kp = max(1, g.shape[1] // max_cols)
    35 +     g = g[:, : g.shape[1] // kp * kp].reshape(g.shape[0], -1, kp).max(2)
    36 +     return {"rows": int(g.shape[0]), "cols": int(g.shape[1]),
    37 +             "g": np.round(g * 235).astype(int).ravel().tolist()}
```

signus/pipeline.py +6 −9 · **코드북 31쪽** · 새로 쓸 줄 22, 126, 413–415, 419

```
  1   1 """End-to-end blind demodulation. Two families:
  2   2   linear (PSK/QAM): front-end -> carrier -> baud -> matched filter -> timing ->
  3   3                      classify -> fine sync -> [equalizer rescue] -> quality
  4   4   fsk (CPFSK/MSK):  frequency discriminator (gated by constant envelope)
  5   5 Classification runs BEFORE fine sync so the decision-directed loop knows the
  6   6 constellation. Differential demapping is an option, not a detection: dqpsk and
  7   7 qpsk share a constellation."""
  8   8
  9   9 import json
 10  10 from dataclasses import dataclass, field
 11  11
 12  12 import numpy as np
 13  13 from scipy.signal import firwin, resample_poly
 14  14
 15  15 from . import classify as cl
 16  16 from . import dsp
 17  17 from .chirp import _bw99, analyze_chirp, is_chirp, sweeps_band
 18  18 from .constellations import demap_bits, demap_diff_bits, mod_order
 19  19 from .eq import equalize, equalize_fse
 20  20 from .fsk import analyze_fsk, fsk_gate
```

```
21 21 from .sigio import Meta, parse_name, read
   22 + from .spectrum import spectrum, strip
23 23 from .sync import find_preamble
24 24
25 25 _BITS_CAP = 65536
26 26 _EQ_LOCK = 60.0    # below this a linear signal is worth an equalizer attempt
27 27 _EQ_GAIN = 3.0     # ...keep it only if lock improves by this much
28 28 _EQ_FLOOR = 50.0   # ...and clears this floor, so a wrong-mod fit is never locked in
29 29 _EQ_QAM_FLOOR = 60.0  # ...but a DENSE-QAM (32/64) eq fit must clear a higher bar: the DD loop c
 ↳  ↳   an
30 30 #                      drag a lower-order cloud onto a 32/64 lattice at a marginal lock (~51)
 ↳  ↳   , a
31 31 #                      confident WRONG order. Genuine multipath recoveries clear ~72; the sweep
 ↳  ↳   's
32 32 #                      eq cases are qpsk/16qam (never 32/64 via eq) so this bar is byte-identic
 ↳  ↳   al.
33 33 _BAUD_GAIN = 5.0   # an in-band baud hypothesis must beat the global one by this lock
34 34 _SHORT_LOCK = 60.0  # below this, retry the baud line with fewer blocks (short-burst rescue)
35 35 _SYNC_LOCK = 60.0   # below this, try a repeated-preamble carrier/timing sync (packetized bursts
 ↳  ↳   )
36 36 _ALIAS_MARGIN = 10.0  # a preamble carrier alias must beat the next by this lock, else ambiguous
 ↳  ↳   .
37 37 #                     This is the LOAD-BEARING rotation guard: even when the coarse anchor itse
 ↳  ↳   lf
38 38 #                     aliases and validates a rotation at ~80 lock, that rotation wins by a thi
 ↳  ↳   n
39 39 #                     margin (~8) -- so a 10-lock margin refuses it. Because the margin catche
 ↳  ↳   s it,
40 40 #                     the lock floor can stay modest (78, not 82) -> ~5% more genuine recoverie
 ↳  ↳   s.
41 41 _SYNC_ACCEPT = 78.0  # ...AND clear this absolute lock: below it correct/rotated aliases overlap
 ↳  ↳   . A
42 42 #                     65 floor was tried (to recover more moderate bursts) but an independent
43 43 #                     snr-12 grid found 8psk rotated aliases locking 71-72 in the opened 65-7
 ↳  ↳   8 band
44 44 #                     (carrier off by one baud/L step -> confident garbage bits). Even 78 leak
 ↳  ↳   s a
45 45 #                     a few hard-corner rotations (a separate pre-existing gate limit): hold a
 ↳  ↳   t 78.
46 46 _SYNC_ADIST = 1.5    # ...AND sit within this many alias steps of the coarse est_carrier fc -- a
47 47 #                     soft backstop to the margin: it catches the one rotation the margin misse
 ↳  ↳   s,
48 48 #                     but tuned LOOSE (1.5, not 0.75) so it does not needlessly reject genuine
49 49 #                     large-offset recoveries. The three together give 0 confident-wrong across
50 50 #                     1920 hard-corner bursts at the max recall this gate reaches.
51 51 _DIFF_OF = {"bpsk": "dbpsk", "qpsk": "dqpsk"}  # pi4dqpsk arrives as qpsk -> dqpsk
52 52
53 53
54 54 @dataclass
55 55 class Result:
56 56     meta: Meta
57 57     family: str                   # 'linear' | 'fsk' | 'chirp'
58 58     burst: tuple[int, int]
59 59     fc: float
60 60     baud: float
61 61     mod: str
62 62     lock: float
63 63     symbols: np.ndarray           # aligned symbols (linear) / normalized levels (fsk)
64 64     bits: np.ndarray
65 65     iq_corr: np.ndarray           # exportable corrected baseband
```

```
    burst_x: np.ndarray = field(repr=False, default=None)  # for spectrum views
    symmetry: int = 0
    carrier_ambiguous: bool = False
    baud_conf: float = 0.0
    rolloff: float | None = None
    h: float | None = None          # FSK modulation index
    evm: float = float("nan")
    mer_db: float = float("nan")
    occupied: int = 0
    snr_est: float = float("nan")
    eq_applied: bool = False
    eq_mode: str | None = None      # 'sym' | 'fse'
    alias_resolved: bool = False
    baud_fallback: bool = False
    bursts: list = field(default_factory=list)
    burst_idx: int = 0
    preamble: object = None         # sync.Preamble when a repeated preamble drove the decode
    chirp: dict | None = None       # chirp/CSS characterization (family 'chirp': no symbols)

    def to_json(self, max_points: int = 6000, views: bool = True) -> dict:
        z = np.nan_to_num(self.symbols)   # a dead/DC capture -> NaN symbols -> invalid JSON
        if z.size > max_points:  # keep TIME ORDER: the UI animates this sequence
            z = z[np.linspace(0, z.size - 1, max_points).astype(int)]
        det = {"fc": round(self.fc, 3), "baud": round(self.baud, 3), "mod": self.mod,
               "symmetry": self.symmetry, "carrier_ambiguous": self.carrier_ambiguous,
               "baud_conf": round(self.baud_conf, 1),
               "alias_resolved": self.alias_resolved,
               "baud_fallback": self.baud_fallback}
        det["rolloff"] = None if self.rolloff is None else round(self.rolloff, 3)
        rf = self.meta.rf_center                        # real RF = capture centre + baseband fc
        det["rf_hz"] = None if rf is None else round(rf + self.fc, 3)
        if self.h is not None:
            det["h"] = round(self.h, 3)
        if self.preamble is not None:                   # a repeated preamble drove the sync
            p = self.preamble
            det["preamble"] = {"period": p.period, "cfo_hz": round(p.cfo_hz, 1),
                               "conf": round(p.conf, 2), "start": p.start, "end": p.end}
        if self.chirp is not None:      # chirp/CSS characterization -- no constellation fields
            c = self.chirp
            det["chirp"] = {"mu": _r(c["mu"], 1), "up": bool(c["up"]), "bw": _r(c["bw"], 1),
                            "sf": c["sf"], "rs": _r(c["rs"], 1), "tsym": _r(c["tsym"], 6)}
        doc = {
            "fs": self.meta.fs, "fmt": self.meta.fmt, "dtype": self.meta.dtype,  # dtyp
                e 은 한 줄
            "rf_center": self.meta.rf_center,          # 보고에 필수: lock 0 의 1순위 용의자다

            "family": self.family, "n_samples": int(self.iq_corr.size),
            "burst": {"start": self.burst[0], "end": self.burst[1]},
            "bursts": [{"start": s, "end": e} for s, e in self.bursts],
            "burst_idx": self.burst_idx,
            "detected": det,
            "quality": {"lock": _r(self.lock, 1) or 0.0, "mer_db": _r(self.mer_db),
                        "evm": _r(self.evm, 4), "occupied": int(self.occupied)},
            "snr_est_db": _r(self.snr_est),
            "eq": {"applied": self.eq_applied, "mode": self.eq_mode},
            "constellation": {"i": np.round(z.real, 4).tolist(),
                              "q": np.round(z.imag, 4).tolist()},
            "bits": "".join(map(str, self.bits[:_BITS_CAP])),
        }
        if views and self.burst_x is not None:
```

```
125 125             doc["spectrum"] = spectrum(self.burst_x, self.meta.fs)
126 +               doc["strip"] = strip(self.burst_x, self.meta.fs)
127 127         return doc
128 128
129 129     def save_iq(self, path: str) -> None:
130 130         np.column_stack([self.iq_corr.real, self.iq_corr.imag]).astype("<f4").tofile(path)
131 131
132 132     def save_symbols(self, path: str) -> None:
133 133         np.save(path, self.symbols.astype(np.complex64))
134 134
135 135     def save_bits(self, path: str, packed: bool = False) -> None:
136 136         if packed:
137 137             np.packbits(self.bits).tofile(path)
138 138         else:
139 139             with open(path, "w") as fh:
140 140                 fh.write("".join(map(str, self.bits)))
141 141
142 142     def save_report(self, path: str) -> None:
143 143         with open(path, "w") as fh:
144 144             json.dump(self.to_json(), fh, indent=1)
145 145
146 146
147 147 def _r(v: float, nd: int = 2) -> float | None:
148 148     return None if v is None or not np.isfinite(v) else round(float(v), nd)
149 149
150 150
151 151 def _fine(z: np.ndarray, mod: str) -> np.ndarray:
152 152     """Bulk-phase removal, decision-directed carrier loop, final rotation lock."""
153 153     return cl.align(dsp.ddsync(cl.align(z, mod), mod), mod)
154 154
155 155
156 156 def _blocks(n: int) -> int:
157 157     """M-th-power spectra are averaged INCOHERENTLY, so a weak tone (high symmetry,
158 158     low SNR) wants fewer, longer segments; only long records need more for
159 159     stationarity. Worth ~2 dB at the 8PSK edge over a fixed blocks=4."""
160 160     return int(np.clip(n // 30000, 1, 8))
161 161
162 162
163 163 def _demod(xd: np.ndarray, fs: float, baud: float, symmetry: int) -> tuple:
164 164     """Matched filter + timing + classify + fine sync at one baud hypothesis."""
165 165     alpha = dsp.est_rolloff(xd, fs, baud)
166 166     ym = dsp.matched(dsp.to_sps(xd, fs, baud), 4, alpha)
167 167     raw = dsp.timing(ym, 4)
168 168     mod = cl.classify(raw, symmetry)
169 169     # align BEFORE ddsync: the coarse carrier leaves only micro-Hz residual, so removing
170 170     # the bulk phase first kills the loop transient that would corrupt the first symbols
171 171     syms = _fine(raw, mod)
172 172     return alpha, ym, raw, mod, syms, cl.quality(syms, mod)
173 173
174 174
175 175 def _rescue(eq_fn, z: np.ndarray, seed_mod: str, symmetry: int) -> tuple:
176 176     """Equalize with a seed constellation, then let the OPENED eye pick the mod:
177 177     heavy ISI corrupts the ring test and even the 2/8 symmetry vote, so re-classify
178 178     with the estimated symmetry AND the neutral order-4 gate, keep the best lock."""
179 179     z1 = eq_fn(z, seed_mod)
180 180     cands = []
181 181     for m in {cl.classify(z1, symmetry), cl.classify(z1, 4)}:   # symmetry too, per the docstrin
    ↳ ↳   g
182 182         s = _fine(z1 if m == seed_mod else eq_fn(z, m), m)
183 183         cands.append((cl.quality(s, m), s, m))
```

```
    best = max(cands, key=lambda c: c[0].lock)
    # Occam de-fold for the PSK point-doubling: CMA is modulus-only, so a bpsk signal under a
    # multipath echo ALSO fits a qpsk ring at a marginally higher lock (phantom 2->4 points).
    # A bpsk candidate that clears the accept floor cannot be an over-fit of qpsk, so prefer it.
    # Only fires when symmetry==2 put bpsk in the set; a genuine qpsk reads symmetry 4 -> the
    # candidate set is the qpsk singleton -> this branch is unreachable and it stays untouched.
    lo = [c for c in cands if c[2] == "bpsk" and c[0].lock >= _EQ_FLOOR]
    if lo and best[2] == "qpsk":
        return lo[0]
    return best


def analyze(x: np.ndarray, meta: Meta, diff: bool = False,
            burst: int | None = None) -> Result:
    """Run the blind chain. `diff` demaps phase transitions (D-BPSK/D-QPSK);
    `burst` selects one detected burst (default: the most energetic)."""
    if x.size:
        # pathological UNIFORM scale must be fixed BEFORE analytic(): its magnitude sanitizer
        # zeroes samples above 1e150 sample-WISE, so a whole capture near/over that ceiling was
        # wiped wholesale (garbage mods, lock=nan at 1e154) before the rms-normalize below could
        # ever help. The 99th percentile is robust to spike outliers (a single corrupt 1e300
        # sample leaves it at signal scale -> untouched here, still zeroed sample-wise later);
        # normal captures sit far inside (1e-100, 1e100) -> untouched -> sweep byte-identical.
        scale = float(np.percentile(np.abs(x[::16]), 99))  # strided: scale detection needs the
        #             BULK magnitude, not every sample -- 16x cheaper on a large direct capture
        if np.isfinite(scale) and scale > 0 and not (1e-100 < scale < 1e100):
            x = x / scale
    x = dsp.analytic(x)
    x = x - x.mean()  # DC block: LO leakage otherwise corrupts every estimator
    if x.size < 64:   # empty / trivially short: nothing to estimate, and it crashes downstream
        raise ValueError(f"신호가 너무 짧습니다 ({x.size} 샘플)")
    rms = float(np.sqrt(np.mean(np.abs(x) ** 2)))
    if rms and (rms < 1e-6 or rms > 1e6):  # only PATHOLOGICAL scale: normalize so the scale-var
      iant
        x = x / rms                       # spots (fsk_gate's CV epsilon, est_carrier's x**p ov
          er/
    #                                       underflow) stay invariant. Normal captures rms~O(1
      ) ->
    #                                       untouched -> the acceptance sweep is byte-identical
      .
    bursts = dsp.find_bursts(x, meta.fs)
    if burst is None or not 0 <= burst < len(bursts):
        burst = int(np.argmax([np.sum(np.abs(x[s:e]) ** 2) for s, e in bursts]))
    s, e = bursts[burst]
    xb = x[s:e] if e - s >= 64 else x

    # chirp gate: an FMCW/CSS sweep trips fsk_gate (bimodal IF) and would force-demodulate
    # into confident garbage FSK. sweeps_band keeps real FSK (0/222 pass) out of this branch,
    # so only a genuine band-sweep lands here -- characterized, never constellation-demodulated.
    # The gates read a band-ISOLATED copy: mix to the spectral centroid, low-pass to the
    # 99%-power band, and decimate so the signal fills ~28% of Nyquist. An off-centre
    # narrowband sweep in a wide record otherwise dilutes the IF statistics with
    # out-of-band noise and leaks through as garbage FSK.
    pw = np.abs(np.fft.fft(xb)) ** 2                # sweep centre = spectral centroid
    fr = np.fft.fftfreq(xb.size, 1 / meta.fs)       # (est_carrier means nothing on a mover)
    fcg = float(np.sum(fr * pw) / (np.sum(pw) + 1e-30))
    xg = dsp.mix(xb, meta.fs, fcg)
    bwg = _bw99(xg, meta.fs)
    want = max(1.25 * bwg, 1e-4 * meta.fs)
    d = int(np.clip(0.28 * meta.fs / want, 1, max(1, xg.size // 256)))
```

```
240 240     fsg = meta.fs / d
241 241     xg = np.convolve(xg, firwin(129, max(min(0.9 * fsg / 2, bwg), 1e-4 * meta.fs)
242 242                                 / (meta.fs / 2)), "same")
243 243     if d > 1:
244 244         xg = resample_poly(xg, 1, d)
245 245     if is_chirp(xg, fsg) and sweeps_band(xg, fsg):
246 246         info = analyze_chirp(xg, fsg)
247 247         return Result(meta, "chirp", (s, e), fcg, info["rs"] or 0.0,
248 248                       "lora" if info["sf"] else "chirp", 0.0, np.zeros(0, complex),
249 249                       np.zeros(0, np.uint8), x, burst_x=xb,
250 250                       bursts=bursts, burst_idx=burst, chirp=info)
251 251
252 252     if fsk_gate(xb, meta.fs):
253 253         r = analyze_fsk(xb, meta.fs)
254 254         sym = np.asarray(r["symbols"], dtype=np.complex128)
255 255         return Result(meta, "fsk", (s, e), r["fc"], r["baud"], r["mod"], r["lock"],
256 256                       sym, r["bits"], xb, burst_x=xb, h=r["h"],
257 257                       bursts=bursts, burst_idx=burst)
258 258
259 259     fc0, symmetry, _ = dsp.est_carrier(xb, meta.fs, blocks=_blocks(xb.size))
260 260     fc = dsp.resolve_alias(xb, meta.fs, fc0, symmetry)
261 261     amb = abs(symmetry * fc) > 0.4 * meta.fs
262 262     xd = dsp.mix(xb, meta.fs, fc)
263 263
264 264     baud, conf = dsp.est_baud(xd, meta.fs)
265 265     fell = False
266 266     alpha, ym, raw, mod, syms, q = _demod(xd, meta.fs, baud, symmetry)
267 267     bw = dsp.occupied_bw(xd, meta.fs)
268 268     if bw and not (0.72 * bw <= baud <= 1.05 * bw):
269 269         # the global |x|^2 peak disagrees with the occupied band: below alpha~0.1
270 270         # (and under harsh channels) data junk outruns the true line. Demod the
271 271         # in-band hypotheses too and let lock decide, with a clear margin.
272 272         for lo in (0.80, 0.67):
273 273             b2, c2 = dsp.est_baud(xd, meta.fs, lo=lo * bw, hi=1.02 * bw)
274 274             if abs(b2 - baud) <= 0.01 * b2:
275 275                 continue
276 276             t = _demod(xd, meta.fs, b2, symmetry)
277 277             if t[5].lock > q.lock + _BAUD_GAIN:
278 278                 alpha, ym, raw, mod, syms, q = t
279 279                 baud, conf, fell = b2, c2, True
280 280
281 281     if q.lock < _SHORT_LOCK:
282 282         # short / low-SNR burst rescue: the default 4-block |x|^2 line splits the record
283 283         # too finely and locks a junk peak. Fewer, longer blocks give a stronger line (the
284 284         # carrier _blocks logic, applied to baud). Run both and keep only a clearly-better
285 285         # lock -- so a long, already-locked signal (CORE) never enters here and is untouched.
286 286         for nb in (2, 1):
287 287             b2, c2 = dsp.est_baud(xd, meta.fs, blocks=nb)
288 288             if abs(b2 - baud) <= 0.01 * b2:
289 289                 continue
290 290             t = _demod(xd, meta.fs, b2, symmetry)
291 291             if t[5].lock > q.lock + _BAUD_GAIN:
292 292                 alpha, ym, raw, mod, syms, q = t
293 293                 baud, conf, fell = b2, c2, True
294 294
295 295     eq_applied, eq_mode, pre = False, None, None
296 296     if q.lock < _EQ_LOCK:  # ISI (multipath) is what a phase-only loop cannot fix
297 297         # CMA is modulus-only, so it opens the eye without knowing the constellation.
298 298         qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
299 299         mode = "sym"
```

```
300 300            if qe.lock < max(_EQ_FLOOR, q.lock + _EQ_GAIN):
301 301                # symbol-spaced FIR could not invert the channel; retry T/2-spaced on
302 302                # the clock-tracked 2 sps stream (unaliased spectrum, longer inverse)
303 303                q2, s2, m2 = _rescue(lambda z, m: equalize_fse(z, m),
304 304                                     dsp.timing(ym, 4, out=2), mod, symmetry)
305 305                if q2.lock > qe.lock:
306 306                    qe, se, me, mode = q2, s2, m2, "fse"
307 307            floor = _EQ_QAM_FLOOR if me in ("32qam", "64qam") else _EQ_FLOOR
308 308            if qe.lock > q.lock + _EQ_GAIN and qe.lock >= floor:
309 309                syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
310 310        elif mod in ("16qam", "32qam", "64qam"):
311 311            # confident square-QAM at high lock: a benign post-echo can fold a LOWER-order signal
312 312            # onto a QAM lattice with a clean-looking eye, so the low-lock rescue above never fires.
313 313            # Equalize once and accept ONLY if a strictly lower order emerges -- verified never to
314 314            # demote a genuine QAM (0/36 across 16/32/64qam x snr x seeds), so real QAM is untouched
    ↳ ↳              .
315 315            qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
316 316            mode = "sym"
317 317            if mod_order(me) < mod_order(mod) and qe.lock < _EQ_FLOOR:
318 318                # a lower order emerged but the symbol-spaced eye stayed shut (echo > ~2 symbols);
319 319                # retry T/2 fractionally-spaced. Only runs once a de-fold is INDICATED, so a genuine
320 320                # QAM (no lower order) never pays for the FSE.
321 321                qe, se, me = _rescue(lambda z, m: equalize_fse(z, m),
322 322                                     dsp.timing(ym, 4, out=2), mod, symmetry)
323 323                mode = "fse"
324 324            if mod_order(me) < mod_order(mod) and qe.lock >= _EQ_FLOOR:
325 325                syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
326 326
327 327        if q.lock < _SYNC_LOCK:
328 328            # packetized-burst rescue -- LAST, after the equalizer: a repeated preamble pins the
329 329            # carrier/baud/timing from only a few symbols, where the blind M-th-power estimator (nee
    ↳ ↳              ding
330 330            # many symbols to average) fails. Runs only if the eq rescue above did NOT lift the lock
    ↳ ↳              , so
331 331            # a multipath signal (which the eq handles, and whose ISI can fake a short period) never
332 332            # reaches here. Detect the preamble, mix by its CFO, demod the DATA past it, keep-best -
    ↳ ↳              - a
333 333            # random / no-preamble signal yields no plateau or a CFO that does not improve lock an
    ↳ ↳              d is
334 334            # dropped (the sweep stays byte-identical). The preamble period P = L*sps supplies the b
    ↳ ↳              aud
335 335            # (fs*L/P per block length L), the phase slope gives the carrier (+-fs/P resolves the L-
    ↳ ↳              fold
336 336            # alias); every PSK/QAM symmetry is tried -- the right (carrier, baud, sym) locks clean.
337 337            ps = find_preamble(xb, meta.fs, baud_hint=baud)
338 338            if ps is not None:
339 339                # The Schmidl-Cox CFO is ambiguous modulo fs/P = baud/L: an M-PSK carrier off by bau
    ↳ ↳                  d/L
340 340                # rotates a whole constellation position per symbol, so the eye still looks locked b
    ↳ ↳                  ut
341 341                # the bits shift -> a confident WRONG decode if the wrong alias is trusted. The coar
    ↳ ↳                  se
342 342                # M-th-power fc lands within ~1 alias step of the truth even on these short bursts
    ↳ ↳                  , so
343 343                # CENTRE the alias scan on it (k0) and sweep +-3 steps -- this reaches the correct a
    ↳ ↳                  lias
344 344                # for any offset (not just |fc|<baud/2); keep-best over them picks the cleanest eye.
345 345                step = meta.fs / ps.period
346 346                k0 = round((fc - ps.cfo_hz) / step)
347 347                cands = []
```

```
348 348                for b2 in sorted({round(meta.fs * L / ps.period) for L in (3, 4, 6, 8)}):
349 349                    for k in range(k0 - 3, k0 + 4):
350 350                        cfo = ps.cfo_hz + k * step
351 351                        data = dsp.mix(xb, meta.fs, cfo)[ps.end:]
352 352                        if data.size < 256:
353 353                            continue
354 354                        for sm in (2, 4, 8):
355 355                            t = _demod(data, meta.fs, float(b2), sm)
356 356                            cands.append((t[5].lock, cfo, float(b2), sm, t))
357 357                cands.sort(key=lambda c: -c[0])
358 358                # Adjacent aliases both look like clean M-PSK, so lock alone can pick the WRONG on
    ↳ ↳                e by a
359 359                # hair. Require the winner to beat the best DIFFERENT-carrier candidate by a clear g
    ↳ ↳                ap;
360 360                # otherwise the alias is genuinely ambiguous -> leave the honest low-lock result rat
    ↳ ↳                her
361 361                # than emit a confident wrong decode. (Same-carrier baud/sym variants do not compete
    ↳ ↳                .)
362 362                # PRECISION GATE: the alias ambiguity (carrier off by baud/L) is blind-ambiguous -
    ↳ ↳                - a
363 363                # rotated alias still locks AS HIGH AS the truth (both are clean M-PSK), so lock alo
    ↳ ↳                ne
364 364                # cannot separate them (a rotated 8psk alias was measured locking 78 with ber 0.38)
    ↳ ↳                . Two
365 365                # conjunctive conditions favour precision over recall: (1) a strong absolute lock, a
    ↳ ↳                nd
366 366                # (2) the winner sits within _SYNC_ADIST alias steps of the coarse est_carrier fc -
    ↳ ↳                - a
367 367                # baud/L rotation is >=1 step off the true carrier, so a far-from-anchor winner is a
368 368                # rotation. Genuine recoveries have a near-anchor carrier (adist<=0.5, lock 88+); o
    ↳ ↳                n the
369 369                # hardest bursts the anchor itself is unreliable -> both conditions fail -> refuse.
370 370                if cands and cands[0][0] >= _SYNC_ACCEPT and cands[0][0] > q.lock + _EQ_GAIN:
371 371                    top = cands[0]
372 372                    other = next((c for c in cands if abs(c[1] - top[1]) > step / 2), None)
373 373                    anchored = abs(top[1] - fc) <= _SYNC_ADIST * step
374 374                    if anchored and (other is None or top[0] - other[0] >= _ALIAS_MARGIN):
375 375                        alpha, ym, raw, mod, syms, q = top[4]
376 376                        fc, baud, symmetry = top[1], top[2], top[3]
377 377                        eq_applied, eq_mode, pre = False, None, ps
378 378                        # diagnostics must describe the ACCEPTED decode, not the abandoned blind
379 379                        # attempt: the carrier came from the preamble (not resolve_alias -> fc0=fc
380 380                        # keeps alias_resolved False), the baud from the preamble period (no
381 381                        # spectral line -> conf 0, no fallback), and ambiguity follows the new fc.
382 382                        fc0, conf, fell = fc, 0.0, False
383 383                        amb = abs(symmetry * fc) > 0.4 * meta.fs
384 384
385 385        bits = demap_diff_bits(syms, _DIFF_OF[mod]) if diff and mod in _DIFF_OF \
386 386            else demap_bits(syms, mod)
387 387        return Result(meta, "linear", (s, e), fc, baud, mod, q.lock, syms, bits, ym,
388 388                      burst_x=xb, symmetry=symmetry, carrier_ambiguous=amb, baud_conf=conf,
389 389                      rolloff=alpha, evm=q.evm, mer_db=q.mer_db, occupied=q.occupied,
390 390                      snr_est=cl.snr_m2m4(syms, mod), eq_applied=eq_applied, eq_mode=eq_mode,
391 391                      alias_resolved=abs(fc - fc0) > 1.0, baud_fallback=fell,
392 392                      bursts=bursts, burst_idx=burst, preamble=pre)
393 393
394 394
395 395    def analyze_file(path: str, fs: float | None = None, fmt: str | None = None,
396 396                     dtype: str | None = None, endian: str | None = None,
397 397                     bitrev: bool | None = None, diff: bool = False,
```

```
398 398            burst: int | None = None, rf: float | None = None) -> Result:
399 399        """Analyze a file; explicit args override SigMF/filename tokens."""
400 400        from .sigio import parse_sigmf
401 401        m = parse_sigmf(path) or parse_name(path)
402 402        meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
403 403                    endian or m.endian, m.bitrev if bitrev is None else bitrev,
404 404                    rf_center=rf if rf is not None else m.rf_center)
405 405        x, meta = read(path, meta)
406 406        return analyze(x, meta, diff, burst)
407 407
408 408
409 409    # --- web payload ------------------------------------------------------------
410 410
411 411    def analyze_web(x: np.ndarray, meta: Meta, *, burst: int | None = None) -> dict:
412 412        """Payload for POST /api/analyze: the analyze() result as JSON, plus -- on the first
413 +        load of a multi-burst capture -- a whole-record spectrogram strip so the UI can draw
414 +        the burst map. Burst re-selection posts again with ?burst=N and skips the overview
415 +        (the UI keeps the one it already has)."""
416 416        r = analyze(x, meta, burst=burst)
417 417        out = r.to_json()
418 418        if burst is None and len(r.bursts) > 1:
419 +            out["overview"] = {"n": int(x.size), "fs": meta.fs, "strip": strip(x, meta.fs)}
423 420        return out
```

signus/web/style.css +12 −3 · **코드북 46쪽** · 새로 쓸 줄 144, 146, 194, 206-214

```
1  1   :root {
2  2     --bg: #f4f6f8; --card: #fff; --ink: #191f28; --sub: #6b7684;
3  3     --blue: #3182F6; --blue-weak: #e8f1fe; --line: #eef1f4;
4  4     --green: #12b886; --amber: #f59f00; --red: #fa5252;
5  5     --field: #fff; --hover: #f7fafd; --border: #dfe4ea;    /* theme-sensitive surfaces */
6  6     --ok-bg: #e6f7f1; --warn-bg: #fff4e0; --bad-bg: #ffecec;
7  7     --radius: 20px; --shadow: 0 6px 24px rgba(30, 42, 66, .07);
8  8   }
9  9   /* dark mode: follows the OS. The signal canvases are already dark, so they blend in. */
10 10  @media (prefers-color-scheme: dark) {
11 11    :root {
12 12      --bg: #0e131b; --card: #161d28; --ink: #e6ebf2; --sub: #94a1b2;
13 13      --blue-weak: #17273c; --line: #232c39; --field: #1b2330; --hover: #1d2632;
14 14      --border: #2b3543; --ok-bg: #102c24; --warn-bg: #302512; --bad-bg: #321b1e;
15 15      --shadow: 0 6px 24px rgba(0, 0, 0, .38);
16 16    }
17 17  }
18 18
19 19  * { box-sizing: border-box; }
20 20
21 21  body {
22 22    margin: 0; background: var(--bg); color: var(--ink);
23 23    font-family: -apple-system, 'Apple SD Gothic Neo', 'Segoe UI', 'Malgun Gothic', sans-serif;
24 24    line-height: 1.5; -webkit-font-smoothing: antialiased;
25 25  }
26 26
27 27  .wrap { max-width: 580px; margin: 0 auto; padding: 8px 20px 64px; }
28 28
29 29  .topbar { max-width: 580px; margin: 0 auto; padding: 28px 20px 12px; }
30 30  .brand { display: flex; align-items: center; gap: 12px; }
31 31  .logo {
32 32    width: 40px; height: 40px; border-radius: 12px; flex: none;
33 33    background: linear-gradient(135deg, var(--blue), #6aa8ff);
34 34  }
35 35  h1 { margin: 0; font-size: 22px; font-weight: 800; letter-spacing: -.4px; }
```

```
.tagline { margin: 2px 0 0; color: var(--sub); font-size: 13px; }

.card {
  background: var(--card); border-radius: var(--radius); padding: 20px;
  box-shadow: var(--shadow); margin-top: 16px; animation: rise .35s ease both;
}
.card-title { margin: 0 0 12px; font-size: 15px; font-weight: 700; }
.hidden { display: none !important; }

@keyframes rise { from { opacity: 0; transform: translateY(10px); } to { opacity: 1; transform
  : none; } }

/* Dropzone */
.drop {
  text-align: center; cursor: pointer; border: 2px dashed var(--border);
  transition: border-color .18s, background .18s, transform .18s;
}
.drop:hover, .drop:focus-visible { border-color: var(--blue); background: var(--hover); outline
  : none; }
.drop.drag { border-color: var(--blue); background: var(--blue-weak); transform: scale(1.01); }
.drop-icon {
  width: 52px; height: 52px; margin: 6px auto 14px; border-radius: 16px;
  background: var(--blue-weak); position: relative;
}
.drop-icon::after {
  content: ''; position: absolute; inset: 16px 15px; border: 3px solid var(--blue);
  border-radius: 3px; border-top: none; border-right: none;
  transform: rotate(45deg) translate(2px, -2px);
}
.drop-title { margin: 0; font-size: 16px; font-weight: 700; }
.drop-sub { margin: 6px 0 0; color: var(--sub); font-size: 13px; }
code { background: var(--line); padding: 1px 6px; border-radius: 6px; font-size: 12px; }

/* Meta form */
.form-title { margin: 0 0 12px; font-size: 14px; font-weight: 600; }
.form-row { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 12px; }
.form-row label { display: flex; flex-direction: column; gap: 6px; font-size: 12px; color: var(-
  -sub); }
input, select {
  font: inherit; padding: 10px 12px; border: 1px solid var(--border); border-radius: 12px;
  background: var(--field); color: var(--ink);
}
input:focus, select:focus { outline: 2px solid var(--blue); border-color: transparent; }

.btn {
  margin-top: 16px; width: 100%; padding: 13px; border: none; border-radius: 14px;
  background: var(--blue); color: #fff; font-size: 15px; font-weight: 700; cursor: pointer;
  transition: filter .15s, transform .1s;
}
.btn:hover { filter: brightness(1.05); }
.btn:active { transform: scale(.98); }
.btn.ghost { background: var(--blue-weak); color: var(--blue); margin-top: 0; }

/* Loading */
.loading { display: flex; align-items: center; gap: 14px; color: var(--sub); }
.spinner {
  width: 26px; height: 26px; border: 3px solid var(--line);
  border-top-color: var(--blue); border-radius: 50%; animation: spin .8s linear infinite;
}
@keyframes spin { to { transform: rotate(360deg); } }
```

```
93 93
94 94  /* Error */
95 95  .error { display: flex; gap: 14px; align-items: center; border-left: 4px solid var(--red); }
96 96  .err-mark {
97 97    width: 34px; height: 34px; flex: none; border-radius: 50%; background: var(--bad-bg);
98 98    color: var(--red); font-weight: 800; display: grid; place-items: center;
99 99  }
100 100 .err-title { margin: 0; font-weight: 700; }
101 101 .err-msg { margin: 2px 0 0; color: var(--sub); font-size: 14px; }
102 102
103 103 /* Summary + gauge */
104 104 .summary-head { display: flex; align-items: center; gap: 12px; margin-bottom: 16px; }
105 105 .pill { padding: 6px 14px; border-radius: 999px; font-size: 13px; font-weight: 700; }
106 106 .pill.ok { background: var(--ok-bg); color: var(--green); }
107 107 .pill.warn { background: var(--warn-bg); color: var(--amber); }
108 108 .pill.bad { background: var(--bad-bg); color: var(--red); }
109 109 .file-name { margin: 0; font-size: 13px; color: var(--sub); overflow: hidden; text-overflow: ell
    ↳ ↳   ipsis; white-space: nowrap; }
110 110
111 111 .summary-body { display: flex; align-items: center; gap: 24px; }
112 112 .gauge { position: relative; width: 120px; height: 120px; flex: none; }
113 113 .donut { width: 120px; height: 120px; }
114 114 .donut-track { fill: none; stroke: var(--line); stroke-width: 12; }
115 115 .donut-arc {
116 116   fill: none; stroke: var(--blue); stroke-width: 12; stroke-linecap: round;
117 117   transform: rotate(-90deg); transform-origin: center;
118 118   transition: stroke-dashoffset .9s cubic-bezier(.22, .61, .36, 1);
119 119 }
120 120 .gauge-center { position: absolute; inset: 0; display: grid; place-items: center; }
121 121 .gauge-num { font-size: 34px; font-weight: 800; line-height: 1; font-variant-numeric: tabular-nu
    ↳ ↳   ms; }
122 122 .gauge-cap { font-size: 11px; color: var(--sub); letter-spacing: 1px; }
123 123
124 124 .metrics { display: flex; flex-direction: column; gap: 12px; }
125 125 .metric { display: flex; flex-direction: column; gap: 2px; }
126 126 .metric-label { font-size: 12px; color: var(--sub); }
127 127 .tip { cursor: help; text-decoration: underline dotted; text-underline-offset: 3px;
128 128   text-decoration-color: color-mix(in srgb, var(--sub) 45%, transparent); }
129 129 .metric-val { font-size: 20px; font-weight: 700; font-variant-numeric: tabular-nums; }
130 130
131 131 /* Card head with actions: spacing lives on the CONTAINER so the title and the
132 132    action stay vertically centered on the same line */
133 133 .card-head { display: flex; align-items: center; justify-content: space-between;
134 134   gap: 12px; margin-bottom: 12px; }
135 135 .card-head .card-title { margin-bottom: 0; }
136 136 .btn.small { width: auto; margin-top: 0; padding: 8px 14px; font-size: 13px; border-radius: 10px
    ↳ ↳   ; }
137 137 .sel.small {
138 138   font: inherit; font-size: 13px; font-weight: 600; padding: 7px 10px; border-radius: 10px;
139 139   border: 1px solid var(--border); background: var(--field); color: var(--ink);
140 140 }
141 141 .play-row { display: flex; gap: 8px; align-items: center; }
142 142 .hint { margin: 8px 0 0; font-size: 12px; color: var(--sub); }
143 143
144 +   /* Spectrum + spectrogram strip */
145 145 .spec { width: 100%; height: 120px; display: block; border-radius: 12px 12px 0 0; background: #0
    ↳ ↳   d1320; }
146 +   .fall { width: 100%; height: 150px; display: block; border-radius: 0 0 12px 12px; background: #0
    ↳ +   00; image-rendering: pixelated; }
147 147
```

```
/* Constellation + playback */
.const { width: 100%; aspect-ratio: 1; border-radius: 14px; display: block; background: #0d1320
  ; }
.scrub { width: 100%; margin-top: 12px; accent-color: var(--blue); }

/* Batch table */
.table-wrap { overflow-x: auto; }
table.batch { width: 100%; border-collapse: collapse; font-size: 13.5px; }
table.batch th {
  text-align: left; font-size: 12px; color: var(--sub); font-weight: 600;
  padding: 8px 10px; border-bottom: 1px solid var(--line);
}
table.batch td { padding: 11px 10px; border-bottom: 1px solid var(--line);
  font-variant-numeric: tabular-nums; }
table.batch .pill { padding: 3px 10px; font-size: 12px; }
table.batch tbody tr { cursor: pointer; transition: background .12s; }
table.batch tbody tr:hover { background: var(--hover); }
table.batch .fn { font-weight: 600; max-width: 210px; overflow: hidden;
  text-overflow: ellipsis; white-space: nowrap; }

/* Param grid */
.param-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-top: 16px; }
.param { padding: 16px; margin: 0; animation: rise .4s ease both; }
.param-label { font-size: 12px; color: var(--sub); }
.param-val { font-size: 22px; font-weight: 800; margin-top: 4px; letter-spacing: -.3px;
  font-variant-numeric: tabular-nums; }
.param-note { font-size: 11px; color: var(--sub); margin-top: 2px; }
.chips { display: flex; flex-wrap: wrap; gap: 6px; margin-top: 10px; }
.chips:empty { display: none; }   /* no flags -> no ghost margin */
.chip { font-size: 11px; font-weight: 700; padding: 4px 9px; border-radius: 999px; }
.chip.hit { background: var(--ok-bg); color: var(--green); }
.chip.miss { background: var(--bad-bg); color: var(--red); }
.chip.warn { background: var(--warn-bg); color: var(--amber); }

/* Burst selector (chips that act) */
.chip-btn {
  font: inherit; font-size: 11px; font-weight: 700; padding: 4px 10px;
  border-radius: 999px; border: 1px solid var(--border); background: var(--field);
  color: var(--sub); cursor: pointer; transition: color .12s, border-color .12s;
}
.chip-btn:hover { border-color: var(--blue); color: var(--blue); }
.chip-btn.on { background: var(--blue-weak); color: var(--blue); border-color: transparent; }

/* Downloads */
.dl-row { display: flex; gap: 10px; flex-wrap: wrap; }
.dl-row .btn { flex: 1; min-width: 120px; }

/* Burst map: whole-record spectrogram strip + clickable burst boxes */
.wf-wrap { position: relative; }
.wf-wrap .fall { height: 230px; border-radius: 12px; }
.boxes { position: absolute; inset: 0; pointer-events: none; }
.box {
  position: absolute; box-sizing: border-box; border: 1px solid rgba(255, 255, 255, .55);
  border-radius: 0; background: rgba(255, 255, 255, .06); cursor: pointer;
  pointer-events: auto; min-width: 7px; min-height: 12px;
  transition: background .12s, box-shadow .12s;
}
.box:hover, .box:focus-visible {
  background: rgba(255, 255, 255, .18); box-shadow: 0 0 0 1px #fff; outline: none;
}
```

```
207 + .box-num {   /* sa 의 번호 레인: 버스트 위쪽에 항상 보이는 번호 */
208 +   position: absolute; top: 2px; left: 50%; transform: translateX(-50%);
209 +   color: #ebebeb; font: bold 11px ui-monospace, SFMono-Regular, monospace;
210 +   text-shadow: 0 0 3px #000, 0 0 3px #000; pointer-events: none;
211 + }
212 + .box-lane {   /* sa 의 검출 띠: 버스트 구간 아래쪽 흰 띠 */
213 +   position: absolute; left: 0; right: 0; bottom: 2px; height: 4px;
214 +   background: #ebebeb; pointer-events: none;
206 215 }
207 216 .box-lbl {   /* hidden by default (a spectrogram full of labels is unreadable); shown on hover */
208 217   position: absolute; top: -17px; left: -2px; white-space: nowrap; font-size: 10px;
209 218   font-weight: 700; color: #fff; background: rgba(13, 19, 32, .85); padding: 1px 6px;
210 219   border-radius: 6px; pointer-events: none; display: none; z-index: 2;
211 220 }
212 221 .box:hover .box-lbl, .box:focus-visible .box-lbl, .box.sel .box-lbl { display: block; }
213 222 .box.sel { border-color: #fff; box-shadow: inset 0 0 0 1px #fff; }
214 223 .pill.gray { background: var(--line); color: var(--sub); }
215 224 table.batch .fc { font-weight: 600; font-variant-numeric: tabular-nums; }
216 225
217 226 /* Non-digital emitter inline info */
218 227
219 228 @media (max-width: 420px) {
220 229   .form-row { grid-template-columns: 1fr; }
221 230   .param-grid { grid-template-columns: 1fr; }
222 231   .summary-body { gap: 18px; }
223 232 }
224 233
225 234 @media (prefers-reduced-motion: reduce) {
226 235   * { animation: none !important; transition: none !important; }
227 236 }
```

signus/web/app.js +20 −32 · **코드북 51쪽** · 새로 쓸 줄 250, 254, 266-267, 447-461, 463

```
1  1  "use strict";
2  2  const $ = (id) => document.getElementById(id);
3  3  const API = "/api/analyze";
4  4  const FS_RE = /(?:^|[._])fs(\d+(?:\.\d+)?(?:e[+-]?\d+)?)(?:[._]|$)/i;
5  5  const RF_RE = /(?:^|[._])rf(\d+(?:\.\d+)?(?:e[+-]?\d+)?)(?:[._]|$)/i;
6  6  const REDUCED = matchMedia("(prefers-reduced-motion: reduce)").matches;
7  7  const state = { file: null, sidecar: null, meta: null, resp: null, batch: [],
8  8                  burst: null, overview: null };
9  9
10 10 /* --- filename parsing (mirror of sigio.parse_name; read-necessities only) --- */
11 11 const DTYPES = ["i8", "u8", "i16", "u16", "f32", "f64"];
12 12 // baudline-style aliases: <bits>t = two's complement (signed), <bits>o = offset (unsigned)
13 13 const DTYPE_ALIAS = { "8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "
   f64" };
14 14 const FMTS = { cplx: "iq", real: "real", iq: "iq" };   // iq = 옛 밑줄 형식의 이름
15 15 /* <이름>.cplx|real.<샘플레이트>.<16t>.pcm — 샘플레이트는 접두사가 없으므로 포맷 바로
16 16    다음 자리로 찾는다. 옛 밑줄 형식(<이름>_fs…_iq_i16.iq)도 그대로 읽는다. sigio.py 와 규칙이
17 17    같아야 한다: 여기서 갈리면 웹은 되고 CLI 는 안 되는(또는 그 반대) 조용한 불일치가 된다. */
18 18 function parseName(name) {
19 19   const base = name.replace(/^.*\//, "").toLowerCase();
20 20   const mt = FS_RE.exec(base), rt = RF_RE.exec(base), toks = base.split(/[._]/);
21 21   // 진짜 포맷 토막 = "숫자 + 아는 제원 토막"을 데리고 다니는 마지막 후보. 라벨의 real/cplx/
22 22   // u8/be 오염과 점 낀 샘플레이트(2.4e6 -> 2 Hz)를 같은 관문으로 막는다 (sigio.py 와 동일 규칙)
23 23   const spec = (t) => DTYPES.includes(t) || t in DTYPE_ALIAS || t === "be" || t === "bitrev"
24 24                       || t === "pcm" || t.startsWith("rf");
25 25   const hit = (k) => /^\d+$/.test(toks[k + 1] || "") && spec(toks[k + 2] || "");
```

```
  const cands = toks.map((t, k) => (t in FMTS ? k : -1)).filter((k) => k >= 0);
  const hits = cands.filter(hit);
  const i = hits.length ? hits[hits.length - 1] : (cands.length ? cands[0] : -1);
  const tail = i >= 0 ? toks.slice(i + 1) : toks;   // 제원은 포맷 토막 뒤에만 산다
  const dt = tail.find((t) => DTYPES.includes(t) || t in DTYPE_ALIAS);
  return {
    fs: hits.includes(i) ? parseFloat(toks[i + 1]) : (mt ? parseFloat(mt[1]) : null),
    fmt: i >= 0 ? FMTS[toks[i]] : null,
    dtype: dt ? (DTYPE_ALIAS[dt] || dt) : "i16",
    endian: tail.includes("be") ? "be" : "le",
    bitrev: tail.includes("bitrev"),
    rf: rt ? parseFloat(rt[1]) : null,
  };
}
/* SigMF: core:datatype -> our fmt/dtype/endian */
const SIGMF = { cf32: ["iq", "f32"], ci16: ["iq", "i16"], ci8: ["iq", "i8"],
                cu8: ["iq", "u8"], rf32: ["real", "f32"], ri16: ["real", "i16"] };
function metaFromSigmf(doc) {
  const g = doc && doc.global;
  if (!g || !g["core:datatype"]) return null;
  const dt = g["core:datatype"], base = dt.replace(/_[lb]e$/, "");
  if (!(base in SIGMF)) return null;
  const [fmt, dtype] = SIGMF[base];
  const cap = (doc.captures || [])[0] || {};
  return { fs: Number(g["core:sample_rate"]), fmt, dtype,
           endian: dt.endsWith("_be") ? "be" : "le", bitrev: false,
           rf: cap["core:frequency"] != null ? Number(cap["core:frequency"]) : null };
}

/* --- number formatting --- */
function sig(x) {
  const a = Math.abs(x);
  return parseFloat(a >= 100 ? x.toFixed(0) : a >= 10 ? x.toFixed(1) : x.toFixed(2)).toString();
}
function fmtHz(v) {
  if (v == null) return "—";
  const a = Math.abs(v);
  if (a >= 1e6) return sig(v / 1e6) + " MHz";
  if (a >= 1e3) return sig(v / 1e3) + " kHz";
  return sig(v) + " Hz";
}
function fmtRf(v) {   // absolute RF: keep kHz resolution even in the 100s-of-MHz range
  return (v / 1e6).toFixed(3) + " MHz";
}

/* --- ideal constellation points (mirror of constellations.ideal_points) --- */
function idealPoints(mod) {
  if (mod === "bpsk") return [{ i: 1, q: 0 }, { i: -1, q: 0 }];
  if (mod === "qpsk" || mod === "8psk") {
    const m = mod === "qpsk" ? 4 : 8, off = mod === "qpsk" ? Math.PI / 4 : 0, p = [];
    for (let k = 0; k < m; k++) p.push({ i: Math.cos(off + 2 * Math.PI * k / m), q: Math.sin(of
      f + 2 * Math.PI * k / m) });
    return p;
  }
  const n = mod === "16qam" ? 4 : mod === "32qam" ? 6 : 8, lv = [], p = [];
  for (let k = 0; k < n; k++) lv.push(k * 2 - (n - 1));
  for (const qi of lv) for (const ii of lv) {
    if (mod === "32qam" && Math.abs(ii * qi) === 25) continue;  // cross: corners removed
    p.push({ i: ii, q: qi });
  }
```

```
  let e = 0;
  for (const pt of p) e += pt.i * pt.i + pt.q * pt.q;
  const norm = Math.sqrt(e / p.length);
  return p.map((pt) => ({ i: pt.i / norm, q: pt.q / norm }));
}

/* --- file intake: one data file (+sidecar) or many (batch) --- */
async function acceptFiles(list) {
  const files = [...list];
  const metas = files.filter((f) => /\.(json|sigmf-meta)$/i.test(f.name));
  const data = files.filter((f) => !metas.includes(f));
  if (!data.length) return showError("데이터 파일을 찾을 수 없어요.");
  if (data.length > 1) return runBatch(data, metas);

  state.file = data[0];
  state.resp = null;
  state.batch = [];
  state.burst = null;
  state.overview = null;
  // 사이드카는 이름이 같은 캡처의 것만 쓴다 -- 확장자만 보고 집으면 다른 신호의 fs/fmt 로
  // 조용히 읽는다 (일괄 모드는 이미 stem 대조를 한다: 두 모드가 다르면 그게 또 함정이다)
  const mine = (m, ext) => {
    const stem = m.name.toLowerCase().slice(0, -ext.length);
    const dn = state.file.name.toLowerCase();
    return dn.startsWith(stem) && (dn.length === stem.length || dn[stem.length] === ".");
  };
  const sm = metas.find((m) => m.name.toLowerCase().endsWith(".sigmf-meta") && mine(m, ".sigmf-m
    eta"));
  const truth = metas.find((m) => m.name.toLowerCase().endsWith(".json") && mine(m, ".json"));
  state.sidecar = truth ? await readJson(truth) : null;   // client-side only, never sent
  state.meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(state.file.name);
  if (state.meta && state.meta.fs && state.meta.fmt) runFirst();
  else showMetaForm();
}
const readJson = (f) => f.text().then(JSON.parse).catch(() => null);

function showMetaForm() {
  hideAll();
  state.meta = state.meta || {};
  if (state.meta.fs) $("mFs").value = state.meta.fs;
  $("mFmt").value = state.meta.fmt || "";
  $("mDtype").value = state.meta.dtype || "i16";
  show($("metaForm"));
}
$("metaGo").onclick = () => {
  const fs = parseFloat($("mFs").value), fmt = $("mFmt").value;
  if (!(fs > 0) || !fmt) return showError("샘플레이트와 포맷을 입력해주세요.");
  state.meta = { fs, fmt, dtype: $("mDtype").value, endian: "le", bitrev: false };
  runFirst();
};

/* --- server call (contract-frozen) --- */
function query(file, m, burst) {
  const q = new URLSearchParams({ name: file.name });
  if (m.fs) q.set("fs", m.fs);
  if (m.fmt) q.set("fmt", m.fmt);
  if (m.dtype) q.set("dtype", m.dtype);
  if (m.endian) q.set("endian", m.endian);
  if (m.bitrev) q.set("bitrev", "1");
  if (m.rf != null) q.set("rf", m.rf);
```

```
144 144   if (burst != null) q.set("burst", burst);
145 145   return q;
146 146 }
147 147 async function postTo(api, file, m, burst) {
148 148   const res = await fetch(api + "?" + query(file, m, burst), { method: "POST", body: file });
149 149   const data = await res.json();
150 150   if (!res.ok || data.error) throw new Error(data.error || `서버 오류 (${res.status})`);
151 151   return data;
152 152 }
153 153 async function runFirst() {                    // entry: first analyze of a fresh capture
154 154   hideAll();
155 155   $("loadMsg").textContent = "신호를 분석하고 있어요…";
156 156   show($("loading"));
157 157   try {
158 158     const resp = await postTo(API, state.file, state.meta);
159 159     state.overview = resp.overview || null;
160 160     state.resp = resp;
161 161     render(resp);
162 162     if (state.overview) renderBurstMap(state.overview, resp);
163 163   } catch (e) {
164 164     showError(e.message);
165 165   }
166 166 }
167 167 async function analyze() {                     // burst re-selection within a single signal
168 168   hideAll();
169 169   $("loadMsg").textContent = "신호를 분석하고 있어요…";
170 170   show($("loading"));
171 171   try {
172 172     state.resp = await postTo(API, state.file, state.meta, state.burst);
173 173     render(state.resp);
174 174     if (state.overview) renderBurstMap(state.overview, state.resp);
175 175     $("backBatch").classList.toggle("hidden", !state.batch.length);
176 176   } catch (e) {
177 177     showError(e.message);
178 178   }
179 179 }
180 180
181 181 /* --- batch mode --- */
182 182 async function runBatch(data, metas) {
183 183   hideAll();
184 184   show($("loading"));
185 185   const truths = new Map(metas.filter((m) => /\.json$/i.test(m.name))
186 186     .map((m) => [m.name.replace(/\.json$/i, ""), m]));
187 187   const sigmfs = new Map(metas.filter((m) => /\.sigmf-meta$/i.test(m.name))
188 188     .map((m) => [m.name.replace(/\.sigmf-meta$/i, ""), m]));
189 189   const rows = [];
190 190   for (let i = 0; i < data.length; i++) {
191 191     const f = data[i];
192 192     $("loadMsg").textContent = `일괄 분석 중… (${i + 1}/${data.length}) ${f.name}`;
193 193     // same meta precedence as single-file mode: a .sigmf-meta sidecar (matched by stem)
194 194     // beats filename tokens
195 195     const sm = sigmfs.get(f.name.replace(/\.[^.]+$/, ""));
196 196     const meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(f.name);
197 197     let resp = null, err = null;
198 198     try {
199 199       if (!meta.fs || !meta.fmt) throw new Error("파일명에 fs/포맷 정보가 없어요");
200 200       resp = await postTo(API, f, meta);
201 201     } catch (e) { err = e.message; }
202 202     const tf = truths.get(f.name);
203 203     // meta is stored per row: a later burst-chip re-analyze posts with THIS file's meta --
```

```
204 204     // it used to read state.meta, which a fresh batch never set (crash) and a previous
205 205     // single-file run left stale (silent wrong-fs decode)
206 206     rows.push({ file: f, meta, resp, err, sidecar: tf ? await readJson(tf) : null });
207 207   }
208 208   state.batch = rows;
209 209   hideAll();
210 210   renderBatch(rows);
211 211 }
212 212
213 213 function renderBatch(rows) {
214 214   $("batchBody").innerHTML = rows.map((r, i) => {
215 215     if (r.err) return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
216 216       `<td colspan="4" style="color:var(--red)">${r.err}</td></tr>`;
217 217     const d = r.resp.detected, lock = Math.round(r.resp.quality.lock);
218 218     const cls = lock >= 60 ? "ok" : lock >= 40 ? "warn" : "bad";
219 219     return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
220 220       `<td>${d.mod.toUpperCase()}</td><td>${fmtHz(d.fc)}</td><td>${fmtHz(d.baud)}</td>` +
221 221       `<td><span class="pill ${cls}">${lock}</span></td></tr>`;
222 222   }).join("");
223 223   $("batchBody").querySelectorAll("tr").forEach((tr) => {
224 224     tr.onclick = () => {
225 225       const r = state.batch[+tr.dataset.i];
226 226       if (!r.resp) return;
227 227       state.file = r.file; state.meta = r.meta; state.resp = r.resp; state.sidecar = r.sidecar;
228 228       state.burst = null; state.overview = null;
229 229       hideAll();
230 230       render(r.resp);
231 231       const b = $("backBatch");
232 232       b.textContent = "← 목록으로";
233 233       b.onclick = () => { hideAll(); renderBatch(state.batch); };
234 234       b.classList.remove("hidden");
235 235     };
236 236   });
237 237   show($("batchCard"));
238 238 }
239 239 $("backBatch").onclick = () => { hideAll(); renderBatch(state.batch); };
240 240 $("dlCsv").onclick = () => {
241 241   const head = "file,mod,fc_hz,baud_hz,rolloff,lock,mer_db,snr_est_db\n";
242 242   const body = state.batch.filter((r) => r.resp).map((r) => {
243 243     const d = r.resp.detected, q = r.resp.quality;
244 244     return [r.file.name, d.mod, d.fc, d.baud, d.rolloff ?? "", q.lock, q.mer_db,
245 245             r.resp.snr_est_db ?? ""].join(",");
246 246   }).join("\n");
247 247   saveBlob("signus_batch.csv", head + body, "text/csv");
248 248 };
249 249
250 +  /* --- burst map: whole-record spectrogram strip with clickable time-burst boxes --- */
251 251 function renderBurstMap(ov, result) {
252 252   show($("burstMap"));
253 253   const n = ov.n || 1, bs = result.bursts || [];
254 +   paintStrip($("burstFall"), ov.strip);
256 255   const host = $("burstBoxes");
257 256   host.innerHTML = "";
258 257   bs.forEach((b, i) => {
259 258     const left = Math.max(0, b.start / n * 100);
260 259     const dur = (b.end - b.start) / ov.fs;
261 260     const lbl = `버스트 ${i + 1} · ${dur >= 1 ? dur.toFixed(2) + " s" : (dur * 1e3).toFixed(0
    ↳ ↳       ) + " ms"}`;
262 261     const box = document.createElement("button");   // full height (one signal), spans t0..t1 i
    ↳ ↳       n time
```

```
263 262     box.className = "box" + (i === result.burst_idx ? " sel" : "");
264 263     box.style.top = "0%"; box.style.height = "100%"; box.style.left = left + "%";
265 264     box.style.width = Math.min(100 - left, Math.max(0.8, (b.end - b.start) / n * 100)) + "%";
266 265     box.title = lbl;
    266 +   box.innerHTML = `<span class="box-num">${i + 1}</span><span class="box-lane"></span>` +
    267 +     `<span class="box-lbl">${lbl}</span>`;
268 268     box.onclick = () => { state.burst = i; analyze(); };   // re-analyse that burst; map persist
          s
269 269     host.appendChild(box);
270 270   });
271 271 }
272 272
273 273 /* --- rendering --- */
274 274 function statusOf(lock) {
275 275   if (lock >= 60) return ["복조 성공", "ok"];
276 276   if (lock >= 40) return ["부분 복조", "warn"];
277 277   return ["복조 실패", "bad"];
278 278 }
279 279 function render(d) {
280 280   hideAll();
281 281   show($("results"));
282 282   $("burstMap").classList.add("hidden");   // re-shown by renderBurstMap only in multi-burst sin
        gle mode
283 283   const lock = d.quality.lock;
284 284   let [txt, cls] = statusOf(lock);
285 285   if (d.detected.chirp) { txt = "처프 특성 보고"; cls = "warn"; }
286 286   const pill = $("statusPill");
287 287   pill.textContent = txt;
288 288   pill.className = "pill " + cls;
289 289   $("fileName").textContent =
290 290     `${state.file.name} · ${fmtHz(d.fs)} · ${d.fmt === "real" ? "Real PCM" : "IQ"}`;
291 291   $("merVal").textContent = d.quality.mer_db == null ? "—" : d.quality.mer_db.toFixed(1) + " dB"
        ;
292 292   $("evmVal").textContent = d.quality.evm == null ? "—" : (d.quality.evm * 100).toFixed(1) + " %
        ";
293 293   $("snrVal").textContent = snrText(d);
294 294
295 295   const flags = [];
296 296   if (d.family === "fsk") flags.push('<span class="chip warn">FSK 계열 · 주파수 판별</span>');
297 297   if (d.family === "chirp") flags.push('<span class="chip warn">처프/CSS · 특성 판독</span>');
298 298   if (d.eq && d.eq.applied) flags.push(`<span class="chip hit">등화기 적용 · ${
299 299     d.eq.mode === "fse" ? "T/2 분수간격" : "심볼간격"}</span>`);
300 300   if (d.detected.alias_resolved) flags.push('<span class="chip warn">반송파 앨리어스 보정</span>
        ');
301 301   if (d.detected.baud_fallback) flags.push('<span class="chip warn">심볼레이트 대역폭 폴백</span
        >');
302 302   if (d.detected.carrier_ambiguous) flags.push('<span class="chip warn">반송파 모호 · 앨리어
        싱 가능</span>');
303 303   $("flagRow").innerHTML = flags.join("");
304 304
305 305   renderBursts(d);
306 306   animateGauge(lock, cls);
307 307   renderParams(d);
308 308   drawSpectrum(d);
309 309   drawWaterfall(d);
310 310   startPlay(d);
311 311 }
312 312 function snrText(d) {
313 313   const s = d.snr_est_db;
314 314   if (s == null || d.quality.lock < 30) return "—";
```

```
315 315   return (s > 28 ? "≥28" : s.toFixed(1)) + " dB";
316 316 }
317 317
318 318 function renderBursts(d) {
319 319   const row = $("burstRow"), bs = d.bursts || [];
320 320   // when the burst MAP is shown (multi-burst capture) the map replaces the chips
321 321   if (state.overview || bs.length < 2) { row.innerHTML = ""; return; }
322 322   const dur = (b) => {
323 323     const sec = (b.end - b.start) / d.fs;
324 324     return sec >= 1 ? sec.toFixed(2) + " s" : (sec * 1e3).toFixed(1) + " ms";
325 325   };
326 326   row.innerHTML = bs.map((b, i) =>
327 327     `<button class="chip-btn${i === d.burst_idx ? " on" : ""}" data-i="${i}">` +
328 328     `버스트 ${i + 1} · ${dur(b)}</button>`).join("");
329 329   row.querySelectorAll("button").forEach((el) => {
330 330     el.onclick = () => { state.burst = +el.dataset.i; analyze(); };
331 331   });
332 332 }
333 333
334 334 function chip(hit, truthTxt) {
335 335   return `<span class="chip ${hit ? "hit" : "miss"}">정답 ${truthTxt} · ${hit ? "일치" : "불일치
    ↳ ↳     "}</span>`;
336 336 }
337 337 function renderParams(d) {
338 338   const det = d.detected, t = state.sidecar && state.sidecar.truth;
339 339   const near = (a, b, tol) => Math.abs(a - b) <= tol;
340 340   if (det.chirp) {              // 처프/CSS: 성상도 제원 대신 특성 (복조 없음, 판독만)
341 341     const c = det.chirp;
342 342     const crows = [
343 343       ["중심주파수", det.rf_hz != null ? fmtRf(det.rf_hz) : fmtHz(det.fc), null,
344 344         det.rf_hz != null ? `기저대역 ${fmtHz(det.fc)}` : ""],
345 345       ["처프", `${c.up ? "▲ 상승" : "▼ 하강"} · ${(c.mu / 1e9).toFixed(3)} MHz/ms`, null,
346 346         `점유대역폭 ${fmtHz(c.bw)}`],
347 347       ["변조방식", c.sf ? `LoRa 추정 SF${c.sf}` : "처프(FMCW/CSS)", null,
348 348         c.sf ? `심볼레이트 ${fmtHz(c.rs)} · 심볼시간 ${(c.tsym * 1e3).toFixed(2)} ms` : ""],
349 349     ];
350 350     $("paramGrid").innerHTML = crows.map(paramCard).join("");
351 351     return;
352 352   }
353 353   const rows = [
354 354     ["중심주파수", det.rf_hz != null ? fmtRf(det.rf_hz) : fmtHz(det.fc),
355 355       t && t.fc != null && chip(near(det.fc, t.fc, Math.max(300, 0.02 * t.baud)), fmtHz(t.fc)),
356 356       det.rf_hz != null ? `기저대역 ${fmtHz(det.fc)}` : ""],
357 357     ["심볼레이트", fmtHz(det.baud), t && t.baud != null && chip(near(det.baud, t.baud, 0.03 * t.
    ↳ ↳       baud), fmtHz(t.baud)),
358 358       det.baud_conf ? `스펙트럴 라인 강도 ×${Math.round(det.baud_conf)}` : ""],
359 359     ["변조방식", det.mod.toUpperCase(), t && t.mod != null && chip(det.mod === t.mod, t.mod.toUp
    ↳ ↳       perCase()),
360 360       det.symmetry ? `대칭 ${det.symmetry}` : "주파수 레벨"],
361 361   ];
362 362   if (d.family === "fsk") {
363 363     rows.push(["변조지수 h", det.h.toFixed(2),
364 364       t && t.h != null && chip(near(det.h, t.h, 0.15), t.h.toFixed(2)), ""]);
365 365   } else {
366 366     rows.push(["롤오프", det.rolloff.toFixed(2),
367 367       t && t.rolloff != null && chip(near(det.rolloff, t.rolloff, 0.11), t.rolloff.toFixed(2))
    ↳ ↳         , ""]);
368 368   }
369 369   $("paramGrid").innerHTML = rows.map(paramCard).join("");
370 370 }
```

```
371 371
372 372 function paramCard([label, val, hit, note]) {
373 373   return `
374 374     <div class="card param">
375 375       <div class="param-label">${label}</div>
376 376       <div class="param-val">${val}</div>
377 377       ${note ? `<div class="param-note">${note}</div>` : ""}
378 378       ${hit ? `<div class="chips">${hit}</div>` : ""}
379 379     </div>`;
380 380 }
381 381
382 382 function animateGauge(lock, cls) {
383 383   const arc = $("donutArc"), num = $("lockNum"), C = 2 * Math.PI * 52;
384 384   arc.style.stroke = { ok: "#12b886", warn: "#f59f00", bad: "#fa5252" }[cls];
385 385   arc.style.strokeDasharray = C;
386 386   const target = C * (1 - Math.max(0, Math.min(100, lock)) / 100);
387 387   if (REDUCED) { arc.style.strokeDashoffset = target; num.textContent = Math.round(lock); return
      ↳ ↳   ; }
388 388   arc.style.strokeDashoffset = C;
389 389   requestAnimationFrame(() => { arc.style.strokeDashoffset = target; });
390 390   const t0 = performance.now();
391 391   const step = (t) => {
392 392     const k = Math.min(1, (t - t0) / 900);
393 393     num.textContent = Math.round(lock * (1 - Math.pow(1 - k, 3)));
394 394     if (k < 1) requestAnimationFrame(step);
395 395   };
396 396   requestAnimationFrame(step);
397 397 }
398 398
399 399 /* --- canvas helpers --- */
400 400 function fit(cv, hCss) {
401 401   const dpr = window.devicePixelRatio || 1, w = cv.clientWidth || 320;
402 402   const h = hCss || w;
403 403   cv.width = w * dpr; cv.height = h * dpr;
404 404   const g = cv.getContext("2d");
405 405   g.setTransform(dpr, 0, 0, dpr, 0, 0);
406 406   return [g, w, h];
407 407 }
408 408 function pct(a, p) {
409 409   const s = [...a].sort((x, y) => x - y);
410 410   return s[Math.min(s.length - 1, Math.max(0, Math.round(p * (s.length - 1))))];
411 411 }
412 412
413 413 /* --- spectrum --- */
414 414 function drawSpectrum(d) {
415 415   // sa 프로브의 power spectrum 판과 같은 조판: 흑배경, 백색 곡선, 회색(0.35) 점선 격자,
416 416   // 바닥 p5-2 .. 최대+2 dB. 검출된 중심주파수(실선)와 ±baud/2(점선)만 기능선으로 얹는다.
417 417   const [g, w, h] = fit($("specCanvas"), 120);
418 418   g.fillStyle = "#000"; g.fillRect(0, 0, w, h);
419 419   if (!d.spectrum) return;
420 420   const f = d.spectrum.f, db = d.spectrum.db;
421 421   const lo = pct(db, 0.05) - 2, hi = pct(db, 1) + 2;
422 422   const fmin = f[0], fmax = f[f.length - 1];
423 423   const X = (v) => (v - fmin) / (fmax - fmin) * w;
424 424   const Y = (v) => h - (Math.max(lo, Math.min(hi, v)) - lo) / (hi - lo) * (h - 8) - 4;
425 425   g.strokeStyle = "rgba(255,255,255,.35)"; g.setLineDash([1, 5]);
426 426   for (let k = 1; k <= 4; k++) {                  // fs/2 의 k/5 지점 세로 격자 (sa 와 동일)
427 427     const gx = fmin + (fmax - fmin) * k / 5;
428 428     g.beginPath(); g.moveTo(X(gx), 0); g.lineTo(X(gx), h); g.stroke();
429 429   }
```

```
430 430   const det = d.detected, half = det.baud / 2e3;
431 431   g.setLineDash([4, 4]);
432 432   for (const gf of [det.fc / 1e3 - half, det.fc / 1e3 + half]) {
433 433     g.beginPath(); g.moveTo(X(gf), 0); g.lineTo(X(gf), h); g.stroke();
434 434   }
435 435   g.setLineDash([]);
436 436   g.strokeStyle = "rgba(255,255,255,.6)";
437 437   g.beginPath(); g.moveTo(X(det.fc / 1e3), 0); g.lineTo(X(det.fc / 1e3), h); g.stroke();
438 438   g.strokeStyle = "#ebebeb"; g.lineWidth = 1.2; g.beginPath();
439 439   f.forEach((v, k) => (k ? g.lineTo(X(v), Y(db[k])) : g.moveTo(X(v), Y(db[k]))));
440 440   g.stroke();
441 441   g.fillStyle = "rgba(255,255,255,.6)"; g.font = "10px sans-serif";
442 442   g.fillText(sig(fmin) + " kHz", 4, h - 4);
443 443   const tmax = sig(fmax) + " kHz";
444 444   g.fillText(tmax, w - g.measureText(tmax).width - 4, h - 4);
445 445 }
446 446
    447 + /* --- spectrogram strip (sa 프로브의 PNG 와 같은 조판) --- */
    448 + function drawWaterfall(d) { paintStrip($("fallCanvas"), d.strip); }
    449 + function paintStrip(canvas, st) {
    450 +   // 서버가 sa 와 같은 규칙(흑백 0..235, hamming 256/128, p25+10..35dB, 열 최대 풀링)으로
    451 +   // 만든 격자를 해상도 그대로 찍는다. CSS 의 image-rendering: pixelated 가 sa PNG 를
    452 +   // 확대해 보는 것과 같은 방식으로 늘린다 -- 보간 뭉개짐이 없다.
    453 +   const g = canvas.getContext("2d");
    454 +   canvas.width = st && st.cols ? st.cols : 1;
    455 +   canvas.height = st && st.rows ? st.rows : 1;
    456 +   if (!st || !st.cols) { g.fillStyle = "#000"; g.fillRect(0, 0, 1, 1); return; }
    457 +   const img = g.createImageData(st.cols, st.rows);
    458 +   for (let k = 0; k < st.g.length; k++) {
    459 +     const o = k * 4;
    460 +     img.data[o] = img.data[o + 1] = img.data[o + 2] = st.g[k];
    461 +     img.data[o + 3] = 255;
465 462   }
    463 +   g.putImageData(img, 0, 0);
476 464 }
477 465
478 466 /* --- constellation playback (the headline) --- */
479 467 const play = { raf: 0, i: 0, on: false, data: null };
480 468 let CG = null;   // cached constellation geometry
481 469
482 470 function stopPlay() { cancelAnimationFrame(play.raf); play.raf = 0; play.on = false; }
483 471 function setPlayBtn(on) { $("playBtn").textContent = on ? "⏸ 일시정지" : "▶ 재생"; }
484 472
485 473 function startPlay(d) {
486 474   stopPlay();
487 475   play.data = d;
488 476   play.i = 0;
489 477   const n = d.constellation.i.length;
490 478   $("scrub").max = String(Math.max(1, n));
491 479   $("playInfo").textContent = `${n.toLocaleString()} 심볼 · 시간 순서로 재생 (잔광 효과)`;
492 480   drawConstBase(d);
493 481   if (REDUCED) {                       // static full view, no motion
494 482     drawPoints(d, 0, n);
495 483     $("scrub").value = String(n);
496 484     setPlayBtn(false);
497 485     return;
498 486   }
499 487   play.on = true;
500 488   setPlayBtn(true);
501 489   play.raf = requestAnimationFrame(loop);
```

```
}
$("playBtn").onclick = () => {
  if (!play.data) return;
  if (play.on) { stopPlay(); setPlayBtn(false); }
  else { play.on = true; setPlayBtn(true); play.raf = requestAnimationFrame(loop); }
};
$("scrub").oninput = () => {
  if (!play.data) return;
  stopPlay(); setPlayBtn(false);
  play.i = +$("scrub").value;
  drawConstBase(play.data);
  drawPoints(play.data, 0, play.i);    // redraw history up to the scrub point
};

function loop() {
  const d = play.data, n = d.constellation.i.length;
  const step = Math.max(1, Math.round(n / 600) * (+$("speedSel").value));
  const to = Math.min(n, play.i + step);
  fade();                              // phosphor persistence
  drawPoints(d, play.i, to);
  play.i = to >= n ? 0 : to;
  if (play.i === 0) drawConstBase(d);  // loop: clear the trail
  $("scrub").value = String(play.i);
  if (play.on) play.raf = requestAnimationFrame(loop);
}

function drawConstBase(d) {
  const [g, w] = fit($("constCanvas"));
  const fsk = d.family === "fsk";
  const I = d.constellation.i, Q = d.constellation.q;
  const ideal = fsk ? [] : idealPoints(d.detected.mod);
  let lim = 0;
  for (let k = 0; k < I.length; k++) lim = Math.max(lim, Math.abs(I[k]), Math.abs(Q[k]));
  for (const p of ideal) lim = Math.max(lim, Math.abs(p.i), Math.abs(p.q));
  lim = (lim || 1) * 1.12;
  const R = w / 2 * 0.9;
  CG = { g, w, lim, R, map: (re, im) => [w / 2 + re / lim * R, w / 2 - im / lim * R] };

  g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, w);
  g.strokeStyle = "rgba(255,255,255,.06)"; g.lineWidth = 1;
  for (let f = -1; f <= 1; f += 0.5) {
    const [gx] = CG.map(f, 0), [, gy] = CG.map(0, f);
    g.beginPath(); g.moveTo(gx, 0); g.lineTo(gx, w); g.stroke();
    g.beginPath(); g.moveTo(0, gy); g.lineTo(w, gy); g.stroke();
  }
  if (fsk) {                           // frequency levels as dashed bands
    g.strokeStyle = "rgba(255,255,255,.5)"; g.setLineDash([6, 5]);
    for (const lv of [...new Set(I.map((v) => Math.round(v)))].filter((v) => Math.abs(v) <= 8)
      ) {
      const [, yy] = CG.map(0, lv);
      g.beginPath(); g.moveTo(0, yy); g.lineTo(w, yy); g.stroke();
    }
    g.setLineDash([]);
  } else {
    g.strokeStyle = "rgba(255,255,255,.75)"; g.lineWidth = 1.4;
    for (const p of ideal) {
      const [x, y] = CG.map(p.i, p.q);
      g.beginPath(); g.arc(x, y, 6, 0, 7); g.stroke();
    }
  }
```

```
}
function fade() {
  if (!CG) return;
  CG.g.fillStyle = "rgba(13,19,32,.14)";   // translucent wash = phosphor decay
  CG.g.fillRect(0, 0, CG.w, CG.w);
}
function drawPoints(d, from, to) {
  if (!CG || to <= from) return;
  const g = CG.g, I = d.constellation.i, Q = d.constellation.q;
  const fsk = d.family === "fsk";
  const a = Math.max(0.08, Math.min(0.55, 150 / (to - from)));
  g.globalCompositeOperation = "lighter";
  g.fillStyle = `rgba(90,170,255,${a})`;
  for (let k = from; k < to; k++) {
    // FSK has no I/Q plane: spread levels vertically, jitter horizontally for density
    const [x, y] = fsk ? CG.map((k % 89) / 89 - 0.5, I[k]) : CG.map(I[k], Q[k]);
    g.beginPath(); g.arc(x, y, 1.8, 0, 7); g.fill();
  }
  g.globalCompositeOperation = "source-over";
}

/* --- downloads --- */
function saveBlob(name, text, type) {
  const url = URL.createObjectURL(new Blob([text], { type }));
  const a = document.createElement("a");
  a.href = url; a.download = name; a.click();
  URL.revokeObjectURL(url);
}
const stem = () => state.file.name.replace(/\.[^.]*$/, "");
$("copyBits").onclick = async (e) => {   // decoded bitstream -> clipboard, with brief feedback
  const btn = e.currentTarget, was = btn.textContent;
  try { await navigator.clipboard.writeText(state.resp.bits); btn.textContent = "복사됨 ✓"; }
  catch { btn.textContent = "복사 실패"; }
  setTimeout(() => { btn.textContent = was; }, 1400);
};
$("dlBits").onclick = () => saveBlob(stem() + ".bits.txt", state.resp.bits, "text/plain");
$("dlSyms").onclick = () => {
  const c = state.resp.constellation;
  saveBlob(stem() + ".symbols.csv", "i,q\n" + c.i.map((v, k) => `${v},${c.q[k]}`).join("\n"), "t
    ext/csv");
};
$("dlReport").onclick = () =>
  saveBlob(stem() + ".report.json", JSON.stringify(state.resp, null, 2), "application/json");

/* --- view helpers --- */
function show(el) { el.classList.remove("hidden"); }
function hideAll() {
  stopPlay();
  ["metaForm", "loading", "errorCard", "results", "batchCard"]
    .forEach((id) => $(id).classList.add("hidden"));
  $("backBatch").classList.add("hidden");
}
function showError(msg) { hideAll(); $("errMsg").textContent = msg; show($("errorCard")); }

/* --- dropzone wiring --- */
const drop = $("dropCard"), input = $("fileInput");
// reset value first: re-picking the SAME file fires no change event otherwise
const pick = () => { input.value = ""; input.click(); };
drop.onclick = pick;
drop.onkeydown = (e) => { if (e.key === "Enter" || e.key === " ") { e.preventDefault(); pick()
```

```
  ; } };
input.onchange = () => { if (input.files.length) acceptFiles(input.files); };
drop.addEventListener("dragover", (e) => { e.preventDefault(); drop.classList.add("drag"); });
drop.addEventListener("dragleave", () => drop.classList.remove("drag"));
drop.addEventListener("drop", (e) => {
  e.preventDefault(); drop.classList.remove("drag");
  if (e.dataTransfer.files.length) acceptFiles(e.dataTransfer.files);
});
```